



CSE381

INTRODUCTION TO MACHINE LEARNING

SUMMER 2026

MILESTONE 2

Prepared To

Dr Mahmoud Khalil
ENG Hala Shaheen

Prepared By

Nadine Radwan 22P0214
Ahmed Mohamed Elsayed 23P0035
Hisham Mohamed 23P0259

Table of Contents

Introduction	6
1.1 Project Background	6
1.2 Problem Statement	6
1.3 Project Objectives	7
1.4 Phase 2 Scope	7
1.5 Project Notebook	8
2. Phase 1 Recap: Data Preparation, EDA, and Baseline Development	9
2.1 Dataset and Initial Inspection	9
2.2 Feature Reduction.....	10
2.3 Data Preprocessing	10
2.4 Exploratory Data Analysis	11
2.4.1 Class Distribution	11
2.4.2 Gill Size and Mushroom Edibility	11
2.4.3 Bruising and Mushroom Edibility	12
2.4.4 Habitat and Mushroom Edibility	13
2.5 Logistic Regression Baseline	13
2.6 Phase 1 Key Findings	15
3. Phase 2 Experimental Setup.....	16
3.1 Phase 2 Objectives.....	16
3.2 Training and Testing Data	16
3.3 Selected Classifier Families	17
3.4 Cross-Validation Strategy	17
3.5 Evaluation Metrics	18
3.5.1 Accuracy	18
3.5.2 Precision	18
3.5.3 Recall	18
3.5.4 F1-Score	18
3.5.5 Confusion Matrix	18

3.5.6 ROC Curve and ROC-AUC	19
3.6 Experimental Consistency	19
4. Task 5: Model Selection and Implementation	20
4.1 Model Selection Strategy	20
4.2 Logistic Regression	20
4.2.1 Model Description	20
4.2.2 Reason for Selection	20
4.2.3 Model Configuration.....	21
4.2.4 Cross-Validation Result	21
4.3 Decision Tree Classifier	22
4.3.1 Model Description	22
4.3.2 Reason for Selection	22
4.3.3 Model Configuration.....	22
4.3.4 Model Training	22
4.3.5 Cross-Validation Result	23
4.4 K-Nearest Neighbors	23
4.4.1 Model Description	23
4.4.2 Reason for Selection	23
4.4.3 Model Configuration.....	23
4.4.4 Model Training	24
4.4.5 Cross-Validation Result	24
5. Task 6: Performance Evaluation and Comparison	24
5.1 Logistic Regression Results	24
5.2 Decision Tree Results.....	25
5.3 K-Nearest Neighbors Results	27
5.4 Overall Performance Comparison	28
5.5 ROC Curve Comparison	29
5.6 Comparison of Error Types	30
5.7 Performance Interpretation	31

6. Task 7: Error Analysis	32
6.1 Misclassification Counts	32
6.2 Logistic Regression Errors	32
6.3 Decision Tree and KNN Errors	33
6.4 False-Positive and False-Negative Analysis.....	33
6.5 Interpretation of the Misclassified Samples	34
6.6 Safety Implications	35
6.7 Limitations of the Error Analysis.....	35
7. Final Model Selection and Discussion.....	36
7.1 Model Selection Criteria.....	36
7.2 Selection Results.....	36
7.3 Selected Final Model	37
7.4 Comparison of Model Families	37
7.4.1 Logistic Regression	37
7.4.2 Decision Tree.....	38
7.4.3 K-Nearest Neighbors	38
7.4.4 Decision Tree Feature Importance.....	38
7.5 Interpretation of Perfect Test Performance.....	40
7.6 Effect of Feature Reduction	40
7.7 Project Strengths	41
7.8 Project Limitations.....	41
7.9 Future Improvements.....	42
7.10 Discussion Summary	43
8. AI Tool Usage Log.....	44
8.1 Example 1: Preprocessing Research and Clarification	44
8.2 Example 2: Evaluation-Metric Research	45
8.3 Example 3: Model-Family Research.....	45
8.4 Example 4: Result Interpretation and Report Organization	46
8.5 Critical Reflection on AI Usage.....	46

9. Contribution Table	48
10. Conclusion	49
11. References.....	50

Introduction

1.1 Project Background

Machine learning classification involves predicting the category of an observation based on patterns learned from labeled examples. In this project, supervised classification techniques were applied to predict whether a mushroom is edible or poisonous using its observable physical characteristics.

Mushroom edibility represents a binary classification problem in which the two possible target classes are edible and poisonous. Because mushrooms from the two classes may share similar characteristics, reliable prediction requires combining information from multiple attributes rather than relying on a single feature.

The project was completed in two phases. Phase 1 established the data-preparation workflow, conducted exploratory data analysis, and developed a Logistic Regression baseline. Phase 2 expanded the study by implementing Decision Tree and K-Nearest Neighbors classifiers, comparing the three models, and examining their classification errors.

1.2 Problem Statement

The objective of the project is to develop and evaluate machine learning models capable of classifying mushrooms into the following categories:

- e: Edible
- p: Poisonous

The original dataset includes attributes that are strongly associated with the target. The odor and spore-print-color features were therefore removed to prevent the classification problem from becoming overly simple. The classifiers were required to make predictions using the combined information contained in the remaining mushroom characteristics.

The consequences of the two error types are not equal. Predicting an edible mushroom as poisonous may result in rejecting a safe mushroom. In contrast, predicting a poisonous mushroom as edible represents a potentially dangerous false-negative error. Model evaluation therefore considered poisonous-class performance and Confusion Matrix results in addition to overall accuracy.

1.3 Project Objectives

The main objectives of the project were to:

- Inspect the structure and quality of the UCI Mushroom Dataset.
- Identify missing values, duplicated records, and the target-class distribution.
- Remove highly obvious predictors to create a more meaningful classification task.
- Prepare the categorical attributes for machine learning.
- Explore relationships between selected mushroom characteristics and edibility.
- Develop Logistic Regression as a baseline classifier.
- Implement Decision Tree and K-Nearest Neighbors classifiers.
- Evaluate the three models using five-fold cross-validation.
- Compare their performance using consistent test-set metrics.
- Analyze their classification errors, particularly poisonous mushrooms predicted as edible.
- Select the most suitable classifier based on predictive performance and safety-related considerations.

1.4 Phase 2 Scope

Phase 2 continues directly from the processed data and baseline model developed during Phase 1. The same training and testing sets are retained to ensure that all models are evaluated under fair and consistent conditions.

The Phase 2 implementation includes:

- Decision Tree classification
- K-Nearest Neighbors classification
- Five-fold cross-validation
- Test-set performance evaluation
- Accuracy and poisonous-class F1-score comparison
- Confusion Matrix analysis
- ROC curve and ROC-AUC comparison

- Misclassification analysis
- Final model selection

The models are evaluated as academic classification systems using a held-out testing set. The resulting predictions must not be used to determine whether mushrooms found in real environments are safe for consumption.

1.5 Project Notebook

The complete executable notebook containing the preprocessing workflow, exploratory analysis, model implementation, cross-validation, test-set evaluation, error analysis, and Decision Tree feature-importance analysis is available through the following Google Colab link:

https://colab.research.google.com/drive/1gHk7dHS_0-hDRb7unL6lWuB5pIbFG4ar?usp=sharing

2. Phase 1 Recap: Data Preparation, EDA, and Baseline Development

Phase 1 established the data-processing and analytical foundation required for the multi-model comparison conducted in Phase 2. It included dataset inspection, feature reduction, preprocessing, exploratory data analysis, and the development of a Logistic Regression baseline.

2.1 Dataset and Initial Inspection

The project uses the Mushroom Dataset obtained from the UCI Machine Learning Repository. The dataset contains 8,124 samples and 22 categorical input attributes describing mushroom characteristics such as cap shape, gill size, bruising, stalk properties, population, and habitat.

The target variable contains two classes:

- e: Edible
- p: Poisonous

The dataset was loaded directly from the UCI repository using the `ucimlrepo` Python package. The input attributes were stored separately from the target variable to prepare the data for inspection and modeling.

Initial analysis produced the following findings:

Property	Result
Total samples	8,124
Original input features	22
Feature type	Categorical
Duplicated rows	0
Edible samples	4,208
Poisonous samples	3,916

The edible samples represented approximately 51.8% of the dataset, while poisonous samples represented approximately 48.2%. The dataset was therefore considered approximately balanced, and no class-balancing technique was required.

The stalk-root attribute was the only feature containing missing values, with 2,480 missing observations. All other features were complete.

2.2 Feature Reduction

The following two attributes were removed:

- odor
- spore-print-color

These attributes are strongly associated with mushroom edibility and could make the classification task overly simple. Their removal encouraged the classifiers to learn from the combined information available in the remaining characteristics instead of depending heavily on obvious predictors.

After feature reduction, 20 input features remained.

2.3 Data Preprocessing

The reduced dataset was divided into training and testing sets using an 80/20 stratified split with `random_state=42`.

Stratification preserved the original proportions of edible and poisonous mushrooms in both subsets. The resulting split contained:

Subset	Number of Samples
--------	-------------------

Training set	6,499
--------------	-------

Testing set	1,625
-------------	-------

The testing set was reserved for final model evaluation and was not used during model training.

Missing values in stalk-root were handled using mode imputation. Because the feature is categorical, its most frequent category was used as the replacement value. The mode was calculated from the training set only and then applied to both the training and testing sets, reducing the risk of test-data leakage.

All remaining input features were categorical and were converted into numerical binary columns using One-Hot Encoding. The training and testing columns were then aligned to guarantee an identical feature structure.

The final processed datasets contained 98 encoded features and zero missing values:

Dataset Final Shape

Training set 6,499 × 98

Testing set 1,625 × 98

2.4 Exploratory Data Analysis

Four visualizations were created during Phase 1 to investigate the target distribution and the relationships between selected mushroom characteristics and edibility.

2.4.1 Class Distribution

The dataset contained 4,208 edible and 3,916 poisonous mushrooms. The similar class counts confirmed that the dataset was approximately balanced.

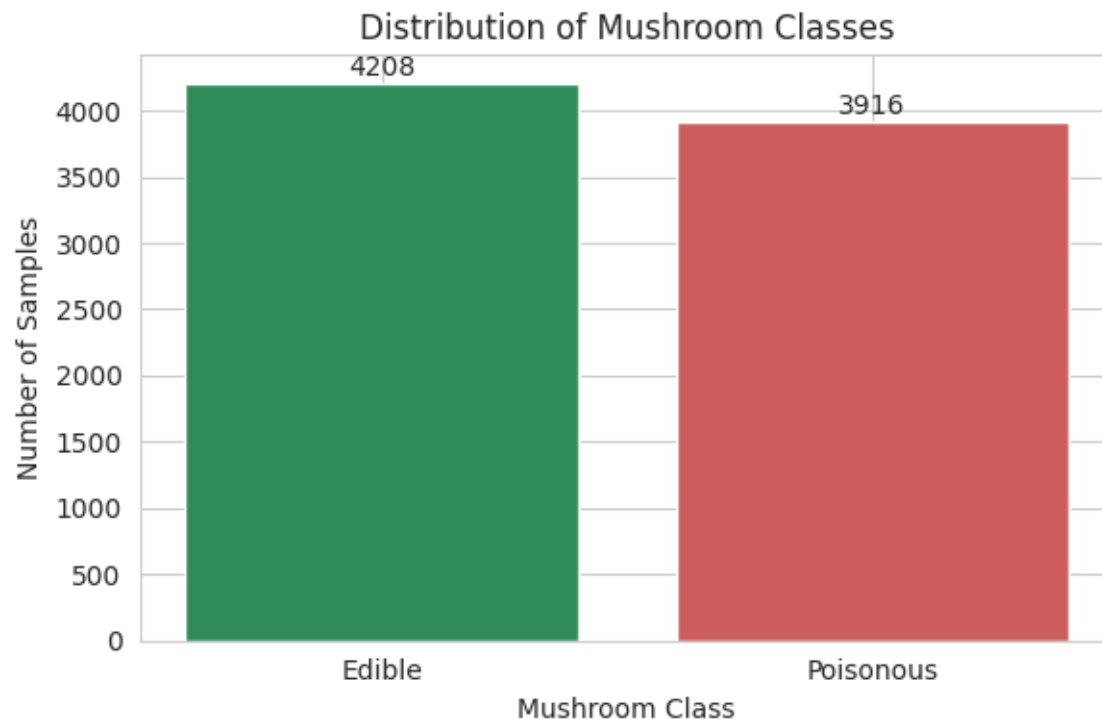


Figure 1: Distribution of edible and poisonous mushrooms.

2.4.2 Gill Size and Mushroom Edibility

Broad gills appeared more frequently among edible mushrooms, while narrow gills were mainly associated with poisonous mushrooms. This difference indicated that gill size provided useful predictive information, although it could not independently determine mushroom edibility.

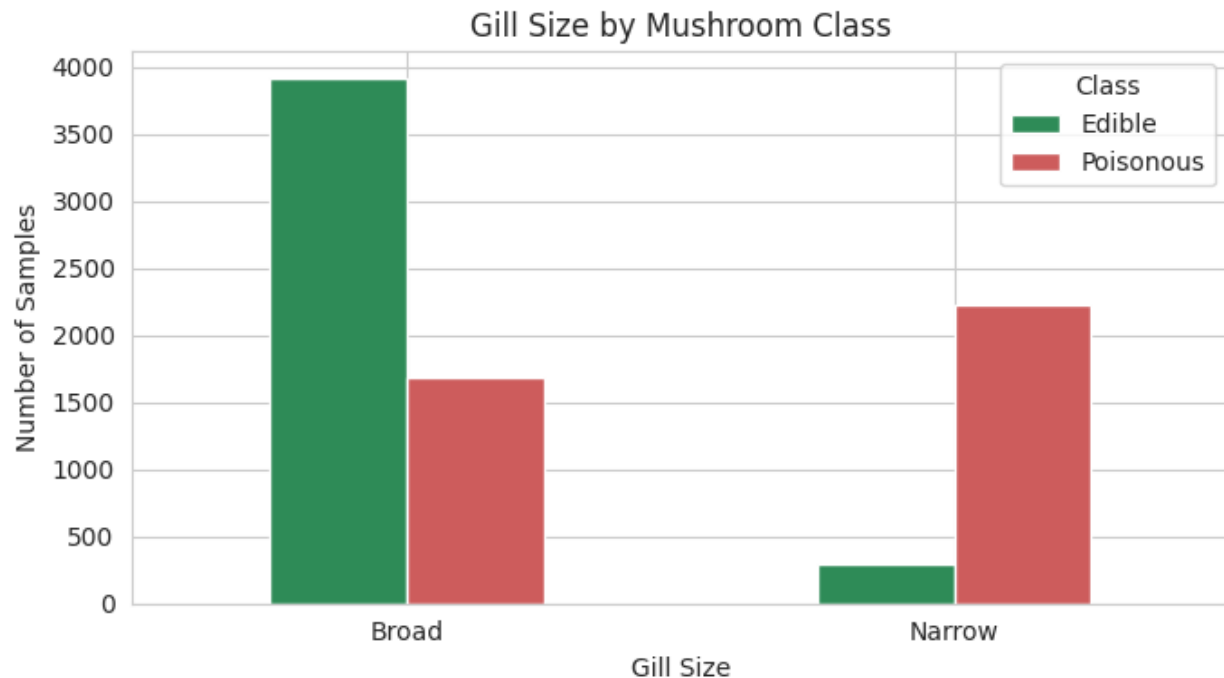


Figure 2: Relationship between gill size and mushroom edibility.

2.4.3 Bruising and Mushroom Edibility

Mushrooms with bruises were more frequently edible, whereas mushrooms without bruises contained a larger number of poisonous samples. Bruising therefore appeared useful for distinguishing between the two target classes.

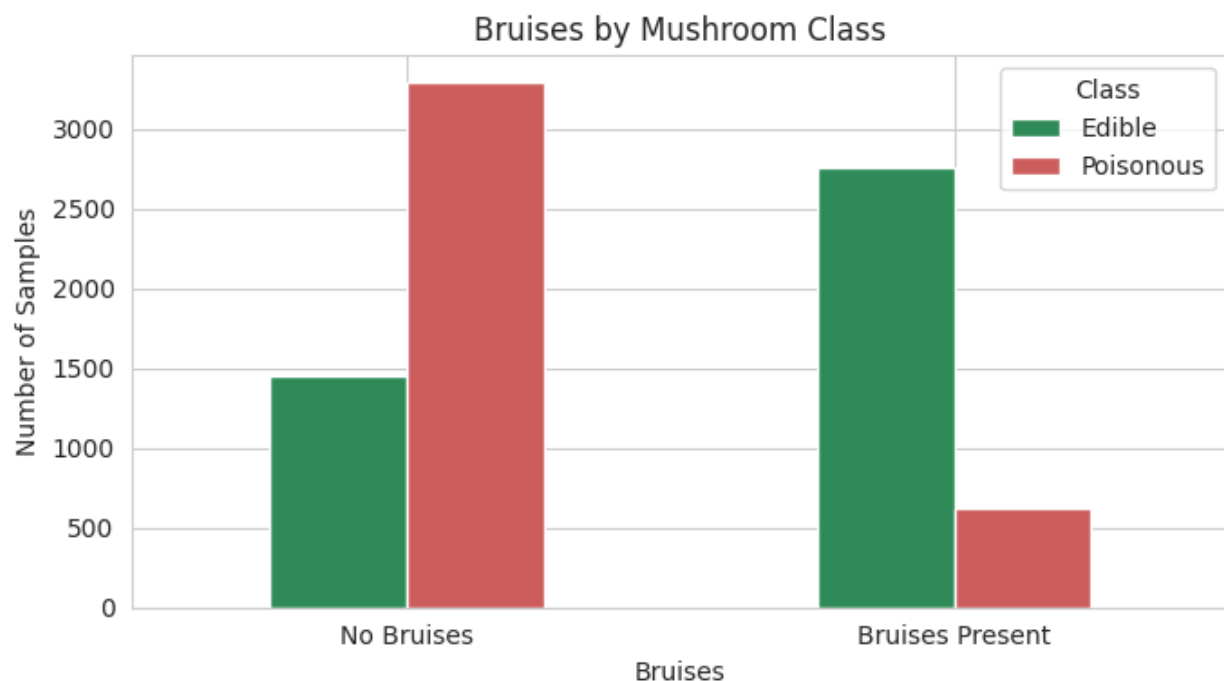


Figure 3: Relationship between bruising and mushroom edibility.

2.4.4 Habitat and Mushroom Edibility

Woods and grasses contained large numbers of mushrooms from both classes but included more edible samples. In contrast, mushrooms found near paths, leaves, and urban environments were more frequently poisonous.

Habitat provided useful contextual information but could not determine mushroom edibility independently.



Figure 4: Distribution of mushroom classes across different habitats.

The exploratory analysis confirmed that meaningful predictive information remained after removing odor and spore-print-color.

2.5 Logistic Regression Baseline

Logistic Regression was selected as the baseline classifier because it is a simple, interpretable, and widely used linear model for binary classification.

The model was created with `max_iter=1000` and evaluated using five-fold cross-validation on the training set. It achieved a mean cross-validation accuracy of approximately 99.49%.

After cross-validation, the model was trained using the complete training set and evaluated on the held-out testing set.

Metric	Result
Mean Cross-Validation Accuracy	99.49%
Test Accuracy	99.57%
Poisonous-Class Precision	100.00%
Poisonous-Class Recall	99.11%
Poisonous-Class F1-Score	99.55%
ROC-AUC	99.996%

The Confusion Matrix contained:

Actual Class	Predicted Edible	Predicted Poisonous
Edible	842	0
Poisonous	7	776

The baseline correctly classified 1,618 of the 1,625 testing samples. However, seven poisonous mushrooms were incorrectly predicted as edible.

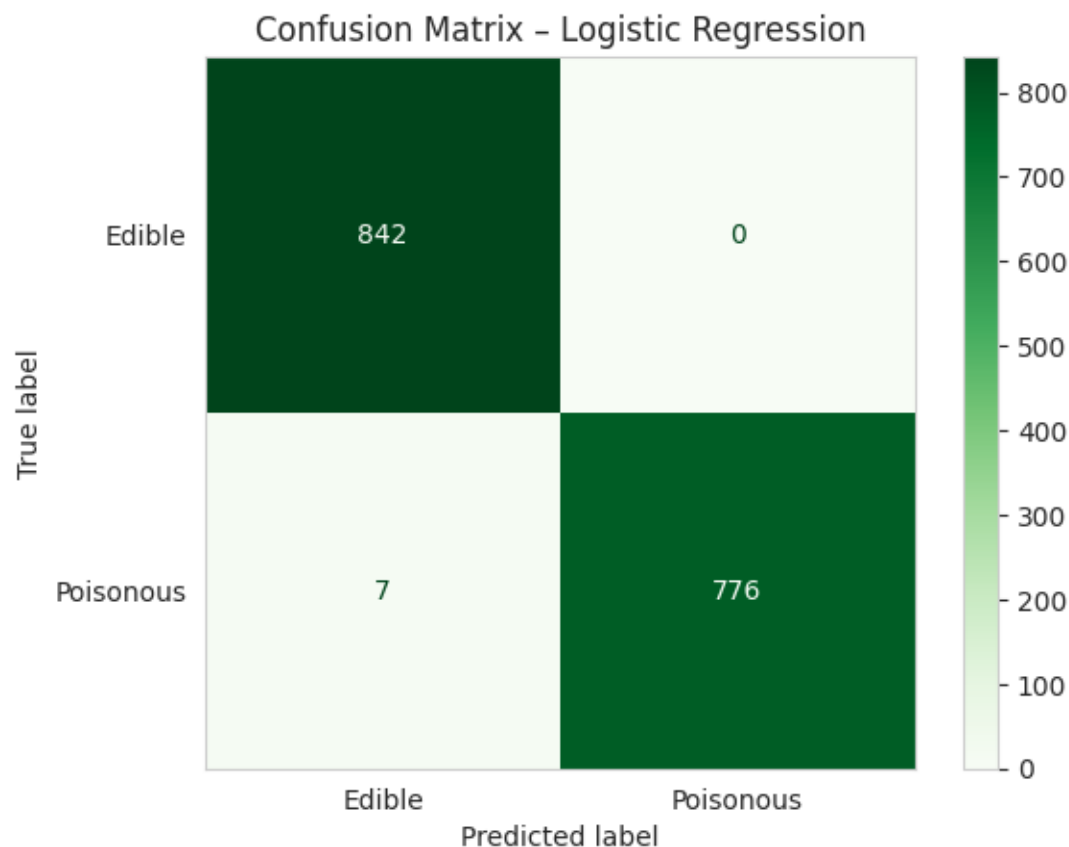


Figure 5: Confusion Matrix for the Logistic Regression baseline.

2.6 Phase 1 Key Findings

Phase 1 produced a complete preprocessing workflow, four focused exploratory visualizations, and a strong Logistic Regression baseline.

The dataset remained highly predictable after removing two strongly associated features. Nevertheless, Logistic Regression produced seven poisonous-to-edible errors. These false-negative predictions established the main motivation for Phase 2: to investigate whether classifiers from other model families could maintain strong overall performance while reducing the most safety-critical errors.

3. Phase 2 Experimental Setup

Phase 2 extended the work completed during Phase 1 by introducing classifiers from additional model families and comparing their performance with the Logistic Regression baseline.

All models were trained and evaluated using the same processed datasets to ensure that the comparison remained fair and consistent.

3.1 Phase 2 Objectives

Phase 2 extended the Logistic Regression baseline developed during Phase 1 by introducing classifiers from two additional model families: Decision Tree and K-Nearest Neighbors.

The main objectives of this phase were to:

- Compare linear, tree-based, and instance-based classification approaches.
- Apply the same training and testing data to all classifiers.
- Evaluate each model using five-fold cross-validation.
- Measure performance on a separate, previously unseen testing set.
- Compare the models using Accuracy, poisonous-class F1-score, Confusion Matrix, and ROC-AUC.
- Identify and examine incorrectly classified samples.
- Give particular attention to poisonous mushrooms incorrectly predicted as edible.
- Determine the most suitable classifier based on performance, consistency, and safety-related errors.

3.2 Training and Testing Data

Phase 2 used the same processed training and testing sets generated during Phase 1. No new split was performed, and no changes were made to the testing samples.

Dataset	Number of Samples	Encoded Features
Training set	6,499	98
Testing set	1,625	98

The training set was used for cross-validation and final model fitting. The testing set remained separate and was used only after training to evaluate the models on unseen samples.

Using an identical testing set for all classifiers ensured that their results could be compared directly.

3.3 Selected Classifier Families

Three classifiers from different model families were included:

Model	Model Family	Main Characteristic
Logistic Regression	Linear	Produces a linear decision boundary in the encoded feature space
Decision Tree	Tree-based	Learns nonlinear rules through feature-based splits
K-Nearest Neighbors	Instance-based	Predicts classes based on nearby training samples

Logistic Regression was retained as the baseline from Phase 1. Decision Tree and KNN were selected to determine whether nonlinear methods could improve the baseline results.

3.4 Cross-Validation Strategy

Five-fold cross-validation was applied separately to each classifier using only the training set.

The procedure followed these steps:

1. Divide the training set into five folds.
2. Train the model using four folds.
3. Validate the model using the remaining fold.
4. Repeat the procedure five times so that each fold is used once for validation.
5. Calculate the mean of the five validation accuracies.

Cross-validation reduces dependence on a single validation split and provides a more reliable estimate of how consistently a model performs across different subsets of the training data.

The held-out testing set was not involved in cross-validation.

3.5 Evaluation Metrics

Several metrics were used because overall accuracy alone does not fully describe classification performance.

3.5.1 Accuracy

Accuracy represents the proportion of all testing samples classified correctly:

Accuracy = Correct Predictions / Total Predictions

Because the dataset is approximately balanced, accuracy provides a useful general measure of performance.

3.5.2 Precision

Poisonous-class precision measures the proportion of mushrooms predicted as poisonous that were actually poisonous:

Precision = $TP / (TP + FP)$

High precision means that the model rarely labels an edible mushroom as poisonous.

3.5.3 Recall

Poisonous-class recall measures the proportion of actually poisonous mushrooms identified correctly:

Recall = $TP / (TP + FN)$

Recall is especially important because a false negative represents a poisonous mushroom incorrectly predicted as edible.

3.5.4 F1-Score

The F1-score combines precision and recall using their harmonic mean:

F1-Score = $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

The poisonous class (p) was explicitly treated as the positive class when calculating the F1-score.

3.5.5 Confusion Matrix

The Confusion Matrix displays the numbers of correct and incorrect predictions for each class. It identifies:

- Edible mushrooms predicted as edible.
- Edible mushrooms predicted as poisonous.

- Poisonous mushrooms predicted as poisonous.
- Poisonous mushrooms predicted as edible.

The last category represents the most safety-critical classification error.

3.5.6 ROC Curve and ROC-AUC

The Receiver Operating Characteristic curve illustrates the relationship between the true-positive rate and false-positive rate across different classification thresholds.

ROC-AUC summarizes the model's ability to separate the two classes:

- A value of 1.0 represents perfect discrimination.
- A value of 0.5 represents performance comparable to random classification.
- Higher values indicate stronger separation between edible and poisonous samples.

ROC-AUC was calculated using the predicted probability of the poisonous class rather than the final predicted class labels.

3.6 Experimental Consistency

The following conditions were kept consistent across all models:

- The same training data.
- The same testing data.
- The same 98 encoded input features.
- The same five-fold cross-validation setting.
- Accuracy as the cross-validation scoring metric.
- The poisonous class as the positive class for F1-score calculation.
- The same evaluation metrics.
- The same testing samples for Confusion Matrix and ROC-AUC calculations.

Maintaining these conditions ensured that the final performance differences represented differences between the classifiers rather than differences in preprocessing or evaluation.

4. Task 5: Model Selection and Implementation

Task 5 extended the Logistic Regression baseline by implementing two additional classifiers from different model families. Decision Tree and K-Nearest Neighbors were selected to provide a meaningful comparison between linear, tree-based, and instance-based classification approaches.

4.1 Model Selection Strategy

The three selected classifiers were:

- Logistic Regression
- Decision Tree
- K-Nearest Neighbors

Each classifier represents a different method of learning the relationship between the input features and mushroom edibility.

Model	Family	Learning Approach
Logistic Regression	Linear	Learns a weighted linear relationship between the encoded features and target
Decision Tree	Tree-based	Learns nonlinear feature-based decision rules
K-Nearest Neighbors	Instance-based	Classifies samples according to nearby training observations

Using classifiers from different families made it possible to examine whether nonlinear and similarity-based methods could improve upon the linear baseline.

4.2 Logistic Regression

4.2.1 Model Description

Logistic Regression is a supervised linear classifier commonly used for binary classification. It estimates the probability that a sample belongs to a particular target class by applying a logistic function to a weighted combination of the input variables.

In this project, the model combined the 98 One-Hot Encoded features to estimate whether each mushroom was edible or poisonous.

4.2.2 Reason for Selection

Logistic Regression was selected during Phase 1 because it is:

- Suitable for binary classification.
- Simple and computationally efficient.
- Capable of generating class probabilities.
- Relatively interpretable.
- A useful baseline for comparison with more flexible classifiers.

The objective of this phase was to compare classifiers from different model families using clear baseline configurations rather than conduct an exhaustive hyperparameter search. Logistic Regression retained its Phase 1 configuration, Decision Tree used the default classifier settings with a fixed random state, and KNN used the standard value of five neighbors. Hyperparameter tuning was therefore identified as a possible future improvement rather than included in the initial model-family comparison.

4.2.3 Model Configuration

The model was implemented using:

`LogisticRegression(max_iter=1000)`

The maximum number of iterations was increased to 1,000 to provide sufficient time for the optimization algorithm to converge. The remaining parameters were retained at their default values because the objective was to establish a clear baseline.

4.2.4 Cross-Validation Result

Five-fold cross-validation produced the following results:

Fold Accuracy

1	99.77%
2	99.85%
3	99.38%
4	99.15%
5	99.31%

Mean 99.49%

The limited variation between the folds indicated that the Logistic Regression baseline performed consistently across different training subsets.

4.3 Decision Tree Classifier

4.3.1 Model Description

A Decision Tree is a supervised learning model that classifies samples through a sequence of feature-based decisions. Each internal node represents a condition applied to a feature, each branch represents the result of that condition, and each leaf represents a predicted class.

Unlike Logistic Regression, a Decision Tree can represent nonlinear relationships and interactions between multiple features.

4.3.2 Reason for Selection

The Decision Tree classifier was selected because it:

- Represents a tree-based model family.
- Can learn nonlinear decision boundaries.
- Captures interactions between mushroom characteristics.
- Does not require feature scaling.
- Can produce understandable feature-based decision rules.
- Provides a useful comparison with the linear baseline.

4.3.3 Model Configuration

The Decision Tree was implemented using:

```
DecisionTreeClassifier(random_state=42)
```

The fixed random state ensured reproducibility. Default values were used for the remaining parameters, meaning that the initial comparison evaluated an untuned Decision Tree.

No maximum depth or pruning parameter was applied. Therefore, the model was allowed to grow until its default stopping conditions were reached.

4.3.4 Model Training

Five-fold cross-validation was first applied using the training set. The classifier was then fitted using all 6,499 training samples and used to predict the classes of the 1,625 testing samples.

Predicted probabilities for the poisonous class were also generated to calculate the ROC-AUC score.

4.3.5 Cross-Validation Result

The Decision Tree achieved a mean five-fold cross-validation accuracy of:

99.9538%

This result was higher than the Logistic Regression mean of 99.4922%. It indicated that the nonlinear rules learned by the Decision Tree were highly suitable for the processed dataset.

The result was close to, but not exactly, 100%. This distinction is important because it shows that a small number of validation samples were misclassified even though the final test-set predictions were perfect.

4.4 K-Nearest Neighbors

4.4.1 Model Description

K-Nearest Neighbors is an instance-based classifier. Instead of learning an explicit equation or tree, it stores the training observations and predicts the class of a new sample according to the classes of its nearest neighbors.

The distance between mushrooms was calculated using their 98 encoded binary characteristics.

4.4.2 Reason for Selection

KNN was selected because it:

- Represents an instance-based model family.
- Can learn nonlinear class boundaries.
- Uses similarity between mushroom samples.
- Provides a different learning approach from both Logistic Regression and Decision Tree.
- Can perform well when samples belonging to the same class form local groups.

4.4.3 Model Configuration

The model was implemented using:

`KNeighborsClassifier(n_neighbors=5)`

The value (k=5) means that each testing sample was classified according to the majority class among its five nearest training observations.

The encoded features were binary, meaning that no feature had a larger numerical scale than the others. The default distance metric and uniform neighbor weighting were retained.

4.4.4 Model Training

Five-fold cross-validation was applied using the training data. The KNN model was then fitted using all training samples and used to classify the testing set.

KNN does not construct a conventional parametric model during fitting. Instead, it stores the training observations and performs most of its computation during prediction.

The proportion of neighboring samples belonging to each class was used to obtain class probabilities for ROC-AUC calculation.

4.4.5 Cross-Validation Result

KNN achieved a mean five-fold cross-validation accuracy of:

99.9538%

This result was approximately equal to the Decision Tree result and higher than the Logistic Regression baseline.

The unrounded mean cross-validation accuracies were:

Model	Mean CV Accuracy
Logistic Regression	0.9949220110
Decision Tree	0.9995381062
KNN	0.9995383431

Although KNN had a marginally higher unrounded value than Decision Tree, the difference was extremely small and did not represent a meaningful practical advantage.

5. Task 6: Performance Evaluation and Comparison

The three classifiers were evaluated on the same held-out testing set using Accuracy, poisonous-class Precision, Recall, F1-score, Confusion Matrix, and ROC-AUC.

Using the same testing samples and evaluation criteria provided a fair comparison between the different model families.

5.1 Logistic Regression Results

Logistic Regression correctly classified 1,618 of the 1,625 testing samples.

Metric	Result
Test Accuracy	99.5692%
Poisonous-Class Precision	100.0000%
Poisonous-Class Recall	99.1054%
Poisonous-Class F1-Score	99.5510%
ROC-AUC	99.9964%

The model achieved very high overall performance. Its poisonous-class precision was 100%, meaning that every mushroom predicted as poisonous was actually poisonous.

However, its recall was approximately 99.11% because it failed to identify seven poisonous samples.

The Logistic Regression Confusion Matrix was:

Actual Class Predicted Edible Predicted Poisonous

Edible	842	0
Poisonous	7	776

No edible mushrooms were predicted as poisonous. However, seven poisonous mushrooms were incorrectly predicted as edible.

These errors are particularly important because they represent the most dangerous type of classification mistake in this application.

5.2 Decision Tree Results

The Decision Tree correctly classified all 1,625 testing samples.

Metric	Result
Test Accuracy	100.0000%
Poisonous-Class Precision	100.0000%
Poisonous-Class Recall	100.0000%
Poisonous-Class F1-Score	100.0000%

Metric	Result
ROC-AUC	100.0000%

The Decision Tree Confusion Matrix was:

Actual Class Predicted Edible Predicted Poisonous

Edible	842	0
Poisonous	0	783

The model correctly identified all 842 edible mushrooms and all 783 poisonous mushrooms. It produced no false-positive or false-negative predictions.

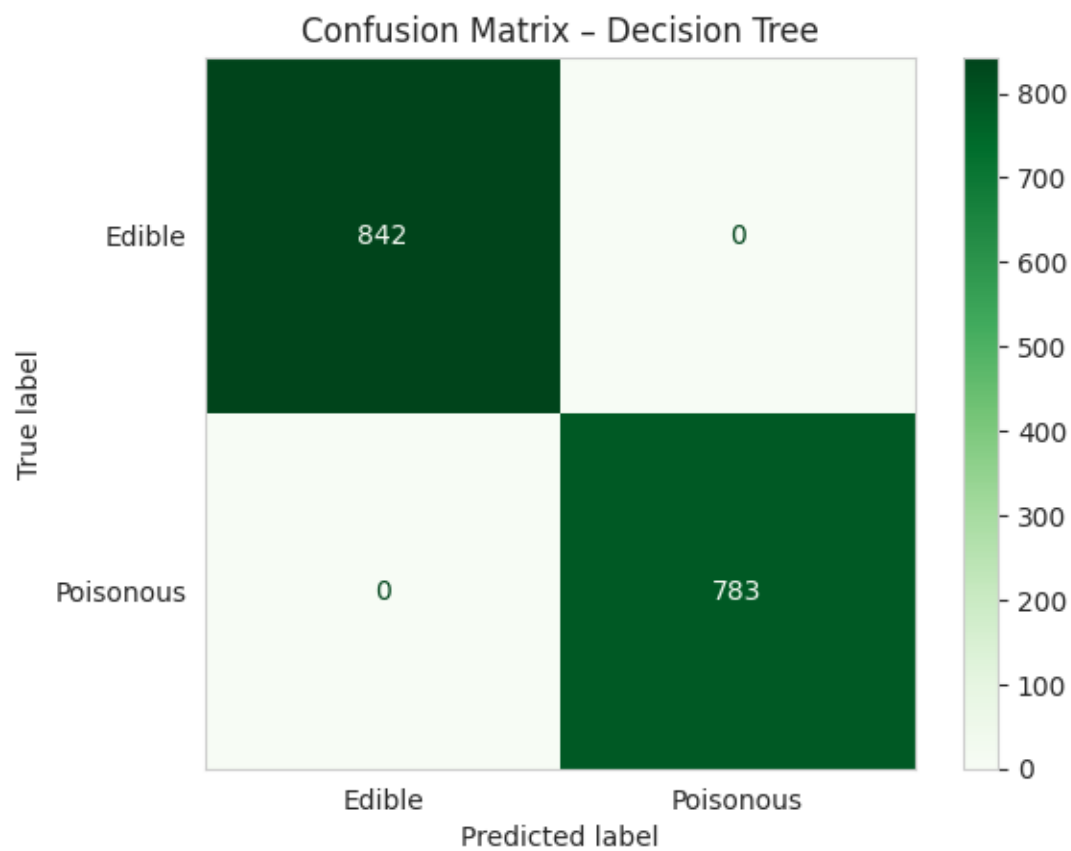


Figure 6: Confusion Matrix for the Decision Tree classifier.

The completely diagonal Confusion Matrix confirms that every testing sample was classified correctly.

5.3 K-Nearest Neighbors Results

KNN also correctly classified all 1,625 testing samples.

Metric	Result
Test Accuracy	100.0000%
Poisonous-Class Precision	100.0000%
Poisonous-Class Recall	100.0000%
Poisonous-Class F1-Score	100.0000%
ROC-AUC	100.0000%

The KNN Confusion Matrix was:

Actual Class	Predicted Edible	Predicted Poisonous
Edible	842	0
Poisonous	0	783

Like the Decision Tree, KNN produced no classification errors.

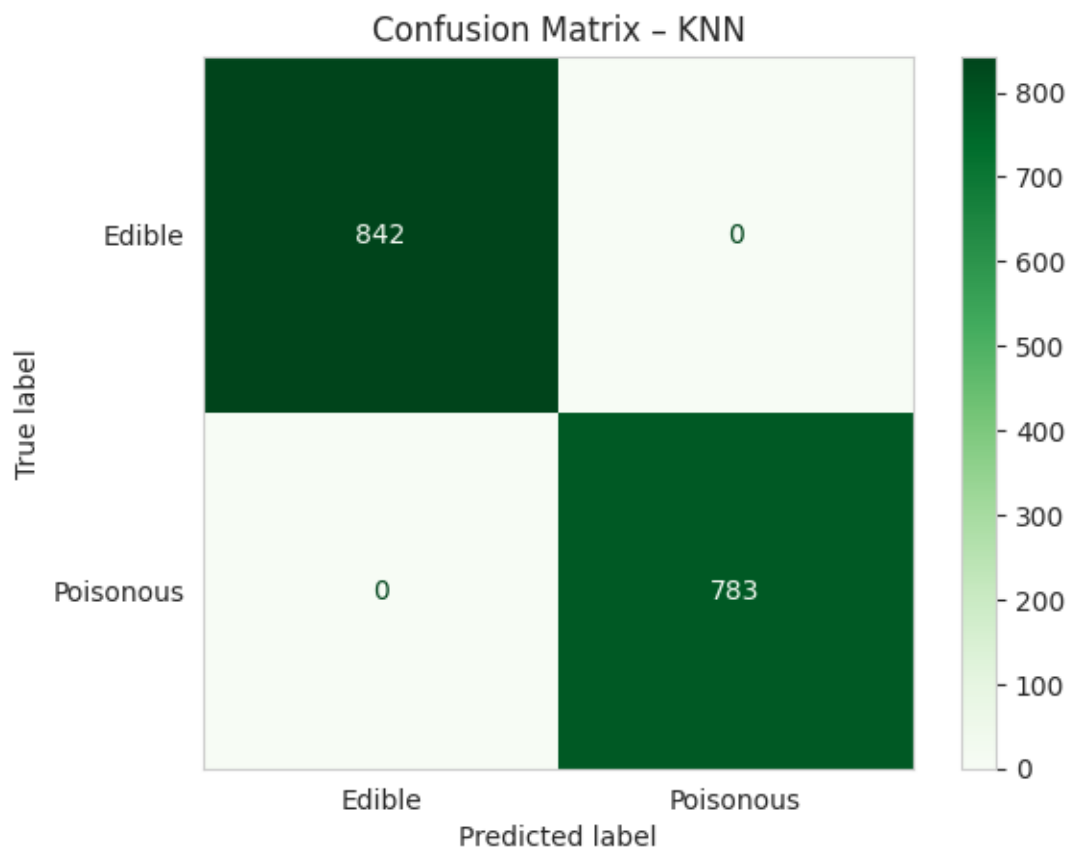


Figure 7: Confusion Matrix for the K-Nearest Neighbors classifier.

The Confusion Matrix confirms that the classifier identified both target classes perfectly on the held-out testing set.

5.4 Overall Performance Comparison

Table 1 summarizes the cross-validation and testing results for all three classifiers.

Model	Mean CV Accuracy	Test Accuracy	Poisonous F1-Score	ROC-AUC
Logistic Regression	99.4922%	99.5692%	99.5510%	99.9964%
Decision Tree	99.9538%	100.0000%	100.0000%	100.0000%
KNN	99.9538%	100.0000%	100.0000%	100.0000%

Table 1: Performance comparison of the three classification models.

All three classifiers achieved strong results. However, Decision Tree and KNN outperformed Logistic Regression across the reported metrics.

Their higher cross-validation accuracy suggests that both nonlinear classifiers performed more consistently across the training folds. Their perfect test accuracy and poisonous-class F1-score also show that they eliminated the seven false-negative errors produced by Logistic Regression.

The unrounded mean cross-validation scores for Decision Tree and KNN were extremely close:

- Decision Tree: 0.9995381062
- KNN: 0.9995383431

The difference is too small to represent a meaningful practical advantage for KNN.

5.5 ROC Curve Comparison

The ROC curves of the three classifiers were plotted together to compare their ability to separate edible and poisonous mushrooms across different decision thresholds.

The models achieved the following ROC-AUC values:

Model	ROC-AUC
Logistic Regression	0.999964
Decision Tree	1.000000
KNN	1.000000

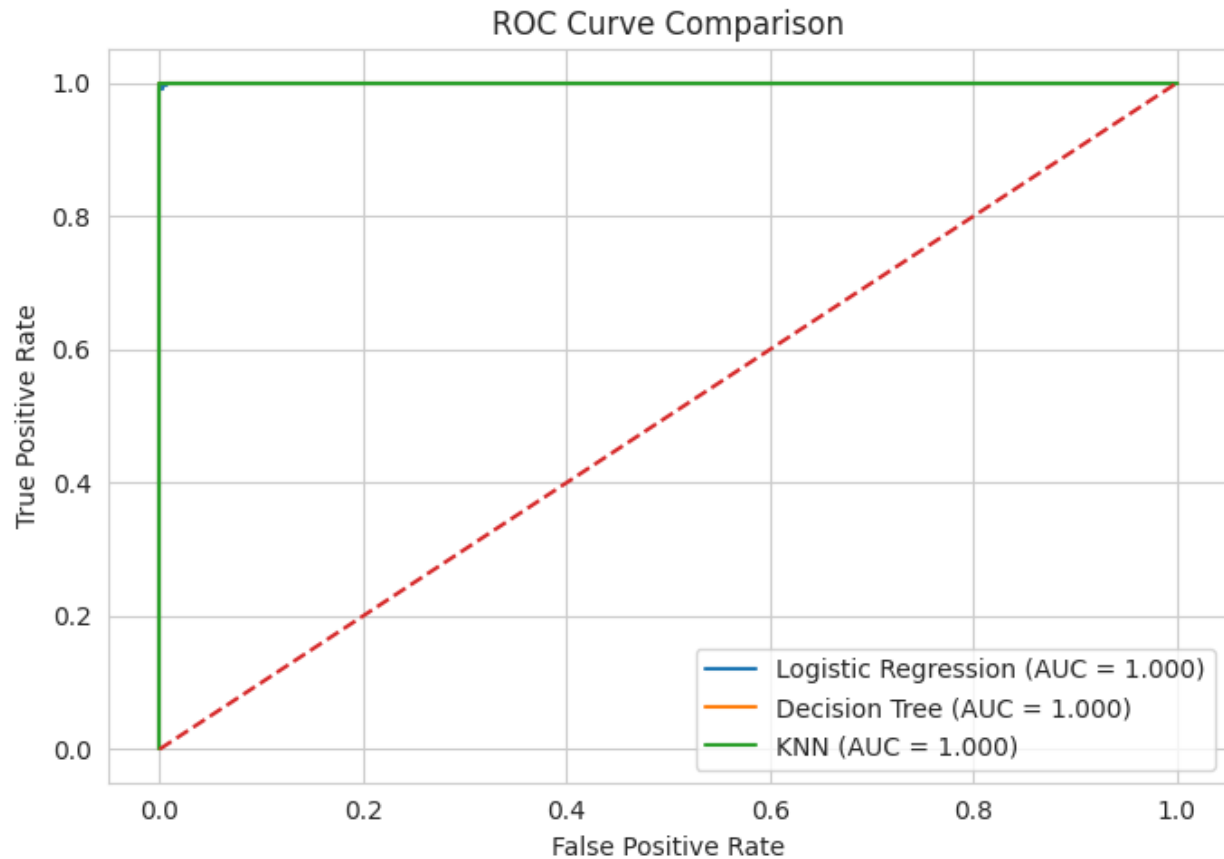


Figure 8: ROC curve comparison for Logistic Regression, Decision Tree, and KNN.

All three ROC curves were located near the upper-left corner, demonstrating excellent class-separation ability.

Decision Tree and KNN achieved an ROC-AUC of 1.0, indicating perfect separation on the testing set. Logistic Regression also achieved a near-perfect ROC-AUC despite producing seven final classification errors.

This demonstrates that ROC-AUC evaluates ranking across multiple thresholds and should be interpreted together with the Confusion Matrix generated using the selected classification threshold.

5.6 Comparison of Error Types

The types of errors produced by each model were:

Model	Edible Predicted Poisonous	Poisonous Predicted Edible	Total Errors
Logistic Regression	0	7	7
Decision Tree	0	0	0
KNN	0	0	0

Logistic Regression produced only poisonous-to-edible errors. Despite representing a small proportion of the testing set, these were the most safety-critical errors.

Decision Tree and KNN correctly classified every sample and produced no errors of either type.

5.7 Performance Interpretation

All three classifiers achieved strong performance, confirming that the remaining mushroom characteristics retained substantial predictive information after removing odor and spore-print-color. Logistic Regression produced seven poisonous-to-edible errors, indicating that its linear decision boundary did not perfectly separate the classes. Decision Tree and KNN captured additional nonlinear relationships and correctly classified every testing sample.

Because Decision Tree and KNN produced identical testing results and nearly identical cross-validation performance, their selection could not be based on predictive metrics alone. Interpretability, prediction efficiency, model complexity, and possible overfitting were therefore considered during final model selection.

6. Task 7: Error Analysis

Error analysis was performed to examine the mistakes produced by each classifier and determine whether the errors followed a meaningful pattern.

Particular attention was given to poisonous mushrooms predicted as edible because these false-negative predictions represent the most serious error in this classification problem.

6.1 Misclassification Counts

The predicted labels produced by each classifier were compared with the actual testing labels.

Model	Testing Samples	Correct Predictions	Incorrect Predictions
Logistic Regression	1,625	1,618	7
Decision Tree	1,625	1,625	0
KNN	1,625	1,625	0

Logistic Regression produced seven errors, while Decision Tree and KNN classified every testing sample correctly.

Task 7: Error Analysis

Error analysis was performed to investigate the mistakes made by the classifiers and identify possible patterns among misclassified samples.

```
1 # Find samples incorrectly classified by Logistic Regression
2 error_mask = y_test != lr_pred
3
4 X_errors = X_test[error_mask].copy()
5 y_errors = y_test[error_mask]
6 pred_errors = lr_pred[error_mask]
7
8 print("Logistic Regression misclassified samples:", error_mask.sum())
9 print("Decision Tree misclassified samples:", (y_test != dt_pred).sum())
10 print("KNN misclassified samples:", (y_test != knn_pred).sum())
```

```
Logistic Regression misclassified samples: 7
Decision Tree misclassified samples: 0
KNN misclassified samples: 0
```

Table 2: Number of correct and incorrect predictions produced by each classifier.

6.2 Logistic Regression Errors

The seven Logistic Regression errors were isolated by comparing `y_test` with the predicted labels.

All seven errors followed the same direction:

Actual Class Predicted Class Number of Samples

Poisonous	Edible	7
-----------	--------	---

Edible	Poisonous	0
--------	-----------	---

The model correctly classified all 842 edible samples. It also correctly identified 776 of the 783 poisonous samples but failed to identify seven poisonous mushrooms.

Its poisonous-class recall was therefore:

$$\text{Recall} = 776 / 783 = 99.11\%$$

The false-negative rate for the poisonous class was:

$$\text{False-Negative Rate} = 7 / 783 = 0.89\%$$

Although this represents a small proportion of the poisonous samples, the direction of the errors makes them important.

6.3 Decision Tree and KNN Errors

Decision Tree and KNN produced zero incorrect predictions on the held-out testing set.

Both classifiers correctly identified:

- All 842 edible mushrooms.
- All 783 poisonous mushrooms.
- All 1,625 testing samples.

Because neither model produced misclassified samples, no false-positive or false-negative patterns were available for further analysis for these two classifiers.

6.4 False-Positive and False-Negative Analysis

The two possible classification errors have different practical consequences.

False Positive

A false positive occurs when an edible mushroom is predicted as poisonous.

This error may result in rejecting a mushroom that is actually edible. Although undesirable, it is the less dangerous error because the model still recommends avoiding the mushroom.

None of the three classifiers produced this error on the testing set.

False Negative

A false negative occurs when a poisonous mushroom is predicted as edible.

This is the most safety-critical error because the prediction incorrectly indicates that a dangerous mushroom is safe.

Logistic Regression produced seven false negatives, while Decision Tree and KNN produced none.

Model	False Positives	False Negatives
Logistic Regression	0	7
Decision Tree	0	0
KNN	0	0

6.5 Interpretation of the Misclassified Samples

The Logistic Regression model applies a linear decision boundary to the One-Hot Encoded feature space. The seven misclassified poisonous mushrooms were therefore assigned to the edible side of this learned boundary.

This suggests that their combined encoded characteristics were more similar to the patterns that the linear model associated with edible samples.

The errors do not necessarily indicate that one individual feature caused the incorrect predictions. Mushroom edibility is represented through combinations of characteristics, and the removed odor and spore-print-color attributes were unavailable to the model.

Decision Tree and KNN classified these samples correctly because they can represent nonlinear relationships:

- Decision Tree can separate samples using sequences of feature-based conditions.
- KNN can classify samples according to nearby observations with similar feature combinations.

This indicates that the errors were more likely related to the limitations of the Logistic Regression decision boundary than to a lack of predictive information in the remaining features.

6.6 Safety Implications

Accuracy alone could make the Logistic Regression result appear almost perfect. However, the Confusion Matrix revealed that every baseline error involved a poisonous mushroom predicted as edible.

This demonstrates why safety-sensitive classification problems should not be evaluated using accuracy alone. Poisonous-class recall, F1-score, and false-negative counts provide essential information about the practical meaning of the model's errors.

According to the testing results, Decision Tree and KNN were safer choices because both achieved 100% poisonous-class recall and produced no false negatives.

However, these results do not guarantee that the models would produce zero false negatives when applied to new data collected from different sources.

6.7 Limitations of the Error Analysis

The error analysis has several limitations:

- It is based on one held-out testing set.
- Only seven Logistic Regression errors were available, limiting the strength of any pattern-based conclusion.
- Decision Tree and KNN produced no test errors, so their possible failure patterns could not be studied.
- One-Hot Encoding expanded the original variables into 98 binary columns, making individual error rows more difficult to interpret directly.
- The dataset may not represent all mushroom varieties or real-world environmental conditions.
- No independent external dataset was available for validation.

Further analysis could decode the seven erroneous rows back into their original categorical characteristics and compare them with correctly classified mushrooms.

7. Final Model Selection and Discussion

This section combines the results of cross-validation, test-set evaluation, and error analysis to identify the most suitable classifier for the project.

7.1 Model Selection Criteria

The final model was selected according to the following criteria:

- Mean cross-validation accuracy.
- Test-set accuracy.
- Poisonous-class precision, recall, and F1-score.
- ROC-AUC.
- Number of false-positive and false-negative predictions.
- Performance consistency.
- Model interpretability.
- Prediction efficiency.
- Risk of overfitting.
- Suitability for a safety-sensitive classification problem.

Accuracy alone was not considered sufficient because a model could achieve high overall accuracy while still classifying poisonous mushrooms as edible.

7.2 Selection Results

The main results were:

Model	CV Accuracy	Test Accuracy	Poisonous Recall	Poisonous F1	False Negatives
Logistic Regression	99.4922%	99.5692%	99.11%	99.5510%	7
Decision Tree	99.9538%	100.0000%	100.00%	100.0000%	0
KNN	99.9538%	100.0000%	100.00%	100.0000%	0

Logistic Regression demonstrated excellent performance but produced seven safety-critical false negatives.

Decision Tree and KNN achieved identical perfect testing results and almost identical cross-validation accuracy. Both eliminated all seven poisonous-to-edible errors produced by the baseline.

7.3 Selected Final Model

The **Decision Tree classifier** was selected as the preferred final model.

Its selection was based on the following findings:

- It achieved 100% test accuracy.
- It achieved a poisonous-class F1-score of 100%.
- It correctly identified all 783 poisonous testing samples.
- It produced zero false-negative predictions.
- It achieved a ROC-AUC of 1.0.
- Its mean cross-validation accuracy was approximately 99.95%.
- It offers clearer feature-based decision rules than KNN.
- It is more efficient during prediction than comparing each new sample with stored training observations.

KNN achieved the same reported testing performance and remains an equally strong predictive alternative. However, the Decision Tree provides a better balance between performance, interpretability, and prediction efficiency.

7.4 Comparison of Model Families

7.4.1 Logistic Regression

Logistic Regression provided a strong and computationally efficient baseline. Its high ROC-AUC indicated excellent separation between edible and poisonous samples.

However, its linear decision boundary did not fully represent all relationships among the encoded mushroom features. This resulted in seven poisonous mushrooms being placed on the edible side of the boundary.

The model remains valuable because it is simple, relatively interpretable, and capable of producing well-structured class probabilities.

7.4.2 Decision Tree

Decision Tree captured nonlinear relationships and interactions between mushroom characteristics. It correctly classified all testing samples and eliminated the baseline's false-negative predictions.

Its feature-based structure can also be inspected to understand how predictions are formed.

However, the tree was implemented using its default depth settings. An unrestricted Decision Tree may learn highly specific patterns and could overfit the training data. Its structure should therefore be examined and controlled before real deployment.

7.4.3 K-Nearest Neighbors

KNN also captured nonlinear relationships and produced perfect testing results. Its performance suggests that samples belonging to the same class formed highly distinguishable local groups in the encoded feature space.

However, KNN stores the training data and performs distance calculations during prediction. This can increase memory usage and prediction time as the dataset grows. Its predictions are also less straightforward to explain than the feature-based decisions of a tree.

7.4.4 Decision Tree Feature Importance

Feature-importance analysis was performed to examine which encoded characteristics contributed most strongly to the predictions made by the selected Decision Tree classifier.

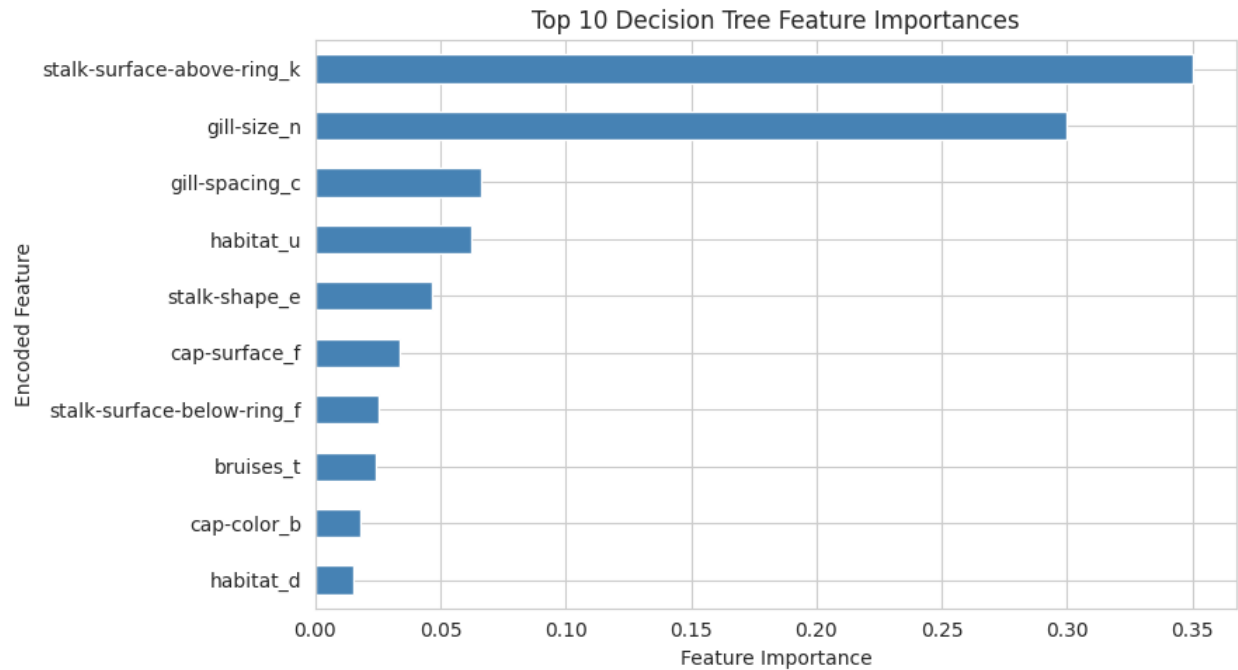


Figure 9: Top ten encoded features according to the fitted Decision Tree classifier.

The most influential encoded feature was stalk-surface-above-ring_k, with an importance of approximately 0.35. This was followed by gill-size_n, with an importance of approximately 0.30. These two encoded features accounted for a substantial proportion of the model's decision-making process.

Other contributing features included gill-spacing_c, habitat_u, stalk-shape_e, cap-surface_f, stalk-surface-below-ring_f, bruises_t, cap-color_b, and habitat_d. Their individual importance values were lower, but they provided additional information used by the tree when separating edible and poisonous samples.

The results show that the Decision Tree combined information from multiple mushroom characteristics rather than relying exclusively on one feature. They also support the exploratory finding that gill size, bruising, and habitat contain meaningful predictive information even after removing odor and spore-print-color.

Feature importance indicates how strongly an encoded feature contributed to the fitted Decision Tree. It does not establish that the characteristic causes a mushroom to be edible or poisonous, and the calculated values may change when the training data or tree configuration changes.

7.5 Interpretation of Perfect Test Performance

The 100% test accuracy achieved by Decision Tree and KNN is supported by their near-perfect cross-validation results. Therefore, the performance is consistent with the strong class structure present in the dataset.

However, perfect test accuracy should be interpreted carefully. It applies only to the selected testing subset from the available UCI dataset.

The result does not prove that the models would achieve perfect performance on:

- Mushroom species not represented in the dataset.
- Samples collected from different geographical regions.
- Incorrectly recorded physical characteristics.
- New categories that were absent during training.
- Noisy or incomplete real-world observations.

The models learned statistical patterns from the dataset rather than biological rules that guarantee mushroom safety.

The perfect testing results should be interpreted as dataset-specific performance rather than evidence of guaranteed real-world classification. The UCI Mushroom Dataset contains structured categorical patterns that may be easier to separate than observations collected under real-world conditions. Furthermore, the testing samples originated from the same dataset as the training samples and may not represent unseen mushroom species, new category values, different measurement conditions, or environmental variations. Therefore, the results demonstrate excellent performance on the available dataset but do not guarantee perfect classification of mushrooms collected from other sources.

7.6 Effect of Feature Reduction

The removal of odor and spore-print-color was intended to prevent the classifiers from relying on two highly obvious predictors.

Despite removing these features:

- Logistic Regression achieved approximately 99.57% test accuracy.
- Decision Tree achieved 100% test accuracy.
- KNN achieved 100% test accuracy.

These results indicate that predictive information was distributed across the remaining mushroom characteristics. The classification task did not depend exclusively on the two removed features.

The strong results also support the EDA findings that gill size, bruising, habitat, and other physical attributes contain useful information when considered together.

7.7 Project Strengths

The main strengths of the project include:

- Use of a real dataset from an established repository.
- Clear separation between input features and the target.
- Stratified train-test splitting.
- Missing-value imputation based on training data only.
- Removal of highly obvious predictors.
- Comparison of three distinct classifier families.
- Consistent five-fold cross-validation.
- Use of the same testing set for all models.
- Evaluation using multiple complementary metrics.
- Explicit focus on poisonous-class performance.
- Analysis of safety-critical false-negative errors.
- Reproducible settings through fixed random seeds.

7.8 Project Limitations

The project has several limitations:

- The Decision Tree and KNN models were not hyperparameter-tuned.
- The Decision Tree was not pruned or depth-limited.
- KNN was evaluated using only (k=5).
- The train-test split was performed once.
- No independent external dataset was available.

- Cross-validation preprocessing was not implemented through a unified scikit-learn Pipeline.
- One-Hot Encoding produced a high-dimensional feature space.
- The seven Logistic Regression errors were not decoded fully into their original categorical values.
- Perfect test performance prevented error-pattern analysis for Decision Tree and KNN.

These limitations do not invalidate the reported results, but they define the conditions under which the results should be interpreted.

Although missing-value imputation was based only on the training set, preprocessing was completed before applying cross-validation. Consequently, the categories used for One-Hot Encoding were identified from the complete training set rather than independently within each fold. This did not expose the models to the held-out testing set, but a stricter implementation would combine preprocessing and classification within a scikit-learn Pipeline so that every transformation is fitted separately inside each cross-validation fold.

The final model was selected after examining the testing results of all three classifiers. Therefore, the reported performance of the selected model may be slightly optimistic. A stricter future study should select the model and its hyperparameters using cross-validation only, then evaluate the selected model once on an independent final testing set.

7.9 Future Improvements

Future work could extend the project through:

- Hyperparameter tuning using Grid Search or Randomized Search.
- Limiting and pruning the Decision Tree.
- Testing different KNN neighbor counts and distance metrics.
- Using a scikit-learn Pipeline to perform preprocessing inside cross-validation.
- Applying repeated or nested cross-validation.
- Examining feature importance for the Decision Tree.
- Decoding Logistic Regression errors into original feature categories.
- Adjusting classification thresholds to prioritize poisonous-class recall.

- Testing additional model families such as Random Forest, Support Vector Machine, or Gradient Boosting.
- Evaluating the selected model using an independent external dataset.
- Measuring training time, prediction time, and memory requirements.

7.10 Discussion Summary

The results show that all three models were highly effective for mushroom classification after preprocessing and feature reduction.

Decision Tree and KNN outperformed Logistic Regression and eliminated all testing errors. Because their numerical performance was effectively identical, practical considerations were required for final selection.

Decision Tree was selected because it combines perfect testing performance with clearer feature-based decisions and efficient prediction. Nevertheless, further validation and complexity control would be necessary before considering its use beyond the academic scope of this project.

8. AI Tool Usage Log

Artificial intelligence was used as a limited research and clarification tool during the project. Its role was restricted to supporting background research, explaining machine learning concepts, identifying relevant documentation, and improving the organization of the written report.

AI was not used as a source of experimental results. All preprocessing outputs, figures, model predictions, evaluation metrics, and comparison results were generated by executing the implemented notebook. The team remained responsible for the methodology, implementation, verification, interpretation, and final conclusions.

8.1 Example 1: Preprocessing Research and Clarification

Research task:

AI assistance was used to research appropriate methods for preparing a dataset composed entirely of categorical variables. The research focused on missing-value treatment, train-test splitting, One-Hot Encoding, and avoiding test-data leakage.

Verification process:

The suggested concepts were reviewed against the characteristics of the UCI Mushroom Dataset and official library documentation.

The implemented workflow was then executed and verified in Google Colab. The inspection confirmed that:

- The dataset contained 8,124 samples and 22 original features.
- stalk-root was the only feature with missing values.
- It contained 2,480 missing observations.
- No duplicated rows were present.
- The target classes were approximately balanced.

The data was split before calculating the imputation value. The stalk-root mode was obtained from the training data only and was then applied to both training and testing sets.

The final encoded training and testing sets contained 98 aligned features and no missing values.

8.2 Example 2: Evaluation-Metric Research

Research task:

AI assistance was used to clarify the differences between Accuracy, Precision, Recall, F1-score, Confusion Matrix, and ROC-AUC.

This research was particularly useful for understanding why overall accuracy should not be used alone in a safety-sensitive classification problem.

Verification process:

The metric definitions and implementations were checked against the official scikit-learn documentation.

The poisonous class was explicitly treated as the positive class when calculating the F1-score because a poisonous mushroom predicted as edible represents the most serious error.

ROC-AUC was calculated using predicted probabilities rather than final class labels. Confusion Matrix values were also checked manually against the number of testing samples.

For Logistic Regression, the values:

- 842 edible mushrooms correctly classified.
- 776 poisonous mushrooms correctly classified.
- Seven poisonous mushrooms classified as edible.
- Zero edible mushrooms classified as poisonous.

sum to 1,625, which matches the complete testing set.

8.3 Example 3: Model-Family Research

Research task:

AI assistance was used to review the general differences between linear, tree-based, and instance-based classifiers.

This supported the selection of three distinct model families:

- Logistic Regression as a linear baseline.
- Decision Tree as a nonlinear tree-based classifier.
- K-Nearest Neighbors as an instance-based classifier.

AI was also used to clarify the meaning of parameters such as `max_iter`, `random_state`, and `n_neighbors`.

Verification process:

The final classifiers and parameter values were selected by the team according to the project requirements and dataset characteristics.

All models were implemented and executed in the notebook. Their results were generated from the same processed data and compared using consistent evaluation metrics.

The final outputs confirmed that Decision Tree and KNN achieved perfect classification on the held-out testing set, while Logistic Regression produced seven errors.

8.4 Example 4: Result Interpretation and Report Organization

Research task:

AI assistance was used to improve the organization of the report and identify a clear structure for presenting Phase 1, model implementation, performance comparison, error analysis, and final model selection.

It was also used to improve the clarity of figure captions and metric explanations.

Verification process:

Every numerical statement was checked against the executed notebook outputs before being included in the report.

The Decision Tree and KNN Confusion Matrices were checked to confirm that each contained:

- 842 correctly classified edible mushrooms.
- 783 correctly classified poisonous mushrooms.
- Zero false positives.
- Zero false negatives.

The final comparison table was also checked against the individual model outputs and ROC-AUC calculations.

Statements that could not be supported directly by the notebook results or official documentation were removed or rewritten.

8.5 Critical Reflection on AI Usage

AI assistance was useful for accelerating background research, clarifying unfamiliar terminology, and organizing technical information into a coherent report structure.

However, AI-generated explanations required review because they could:

- Introduce methods not required by the project.
- Suggest unnecessary implementation complexity.
- Provide interpretations that were not supported by the actual results.
- Use unsuitable preprocessing approaches.
- Present numerical assumptions as if they were executed results.

To manage these limitations, the team verified information using:

- The official UCI dataset description.
- The executed Google Colab notebook.
- Official pandas documentation.
- Official scikit-learn documentation.
- The actual figures, tables, and evaluation outputs.

AI was therefore treated as a research and communication support tool rather than as the creator of the project. The complete workflow was implemented and validated by the team, and all reported results were generated by the project code.

9. Contribution Table

Team Member	Assigned role	Main contribution
Nadine Radwan	Data and Preprocessing Lead	Data acquisition, inspection, cleaning, feature reduction, splitting, missing-value handling, and encoding
Hisham Mohamed	EDA and Visualization Lead	Descriptive analysis, figures, captions, and interpretations
Ahmed Elsayed	Modeling and Evaluation Lead	Baseline modeling, cross-validation, evaluation, and result analysis

10. Conclusion

This project developed and evaluated an end-to-end supervised classification workflow for predicting mushroom edibility using the UCI Mushroom Dataset. After removing odor and spore-print-color, the remaining 20 categorical characteristics were preprocessed into 98 aligned encoded features and used to compare classifiers from three different model families.

Logistic Regression, Decision Tree, and K-Nearest Neighbors were evaluated using five-fold cross-validation and a separate testing set. Logistic Regression achieved approximately 99.57% test accuracy but incorrectly classified seven poisonous mushrooms as edible. Decision Tree and KNN correctly classified all 1,625 testing samples and achieved perfect poisonous-class F1-scores and ROC-AUC values on the held-out test set.

The Decision Tree feature-importance analysis showed that the model combined information from multiple characteristics. The strongest contributors included stalk-surface-above-ring_k and gill-size_n, followed by features related to gill spacing, habitat, stalk shape, cap surface, and bruising.

Decision Tree was selected as the preferred final model. Although its predictive performance was effectively identical to KNN, it offered clearer feature-based decisions and more efficient prediction. KNN remains an equally strong predictive alternative according to the measured results.

The perfect testing results should be interpreted as dataset-specific performance rather than proof of guaranteed real-world classification. The models were evaluated using samples from a single dataset, and the final model was selected after examining the test results. Additional validation, Pipeline-based cross-validation, hyperparameter tuning, and testing on independent data would be required before considering broader use. The developed models are suitable for academic analysis and must not be used to determine whether real mushrooms are safe to consume.

11. References

1. UCI Machine Learning Repository. (1987). *Mushroom [Dataset]*. University of California, Irvine. <https://doi.org/10.24432/C5959T>
2. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
3. Scikit-learn Developers. (n.d.). *LogisticRegression documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
4. Scikit-learn Developers. (n.d.). *DecisionTreeClassifier documentation*. <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
5. Scikit-learn Developers. (n.d.). *KNeighborsClassifier documentation*. <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>
6. Scikit-learn Developers. (n.d.). *Cross-validation: Evaluating estimator performance*. https://scikit-learn.org/stable/modules/cross_validation.html
7. Scikit-learn Developers. (n.d.). *Metrics and scoring: Quantifying the quality of predictions*. https://scikit-learn.org/stable/modules/model_evaluation.html
8. The Pandas Development Team. (n.d.). *Pandas documentation: get_dummies*. https://pandas.pydata.org/docs/reference/api/pandas.get_dummies.html
9. Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees*. Wadsworth International Group.
10. Cover, T. M., & Hart, P. E. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27. <https://doi.org/10.1109/TIT.1967.1053964>
11. McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61). <https://doi.org/10.25080/Majora-92bf1922-00a>
12. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
13. Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), Article 3021. <https://doi.org/10.21105/joss.03021>
14. OpenAI. (2026). *ChatGPT [Large language model]*. <https://chatgpt.com/>